AI 答疑助手优化实践：从 RAG 到 LightRAG 的全链路升级

原创 跨端技术团队 大淘宝技术 2026年4月10日 01:35 浙江 564人

本文针对传统RAG存在的意图识别模糊、知识碎片化及缺乏评测闭环等痛点，提出了一套系统性解决方案：首先，利用**思维链（CoT）驱动的意图识别**，将用户问题分解为多步逻辑查询并行检索，解决了上下文工程中查询不精准的问题；其次，在检索架构上，对比了GraphRAG高昂的构建成本与维护难度，文章重点阐述了**LightRAG**的落地实践，通过实体关系抽取与双层检索范式，在保留图结构优势的同时实现了秒级响应与增量更新；最后，构建了多维度的**评测体系**，强调人工校验以克服模型“过度自信”，旨在通过数据驱动的方式持续提升答疑系统的上下文构建能力。

01 引言

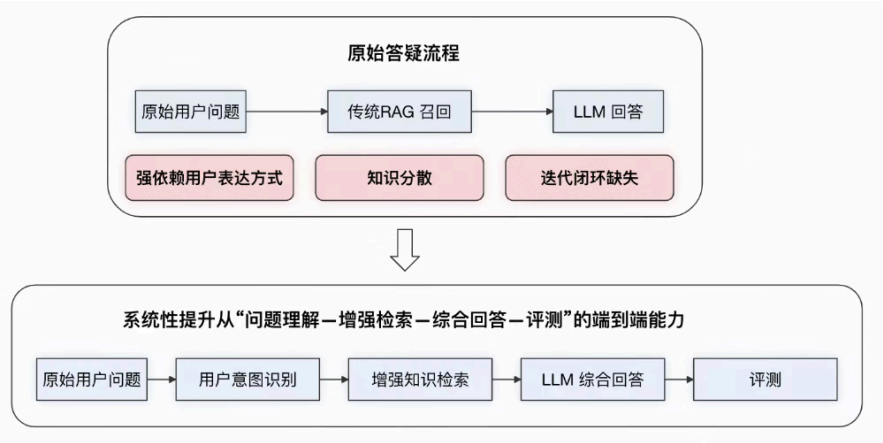
当你问一个 AI 助手"怎么在平台上配置灰度发布"，它却回答你一段 Kubernetes 的通用教程——这种"答非所问"的体验，相信很多做过 AI 答疑系统的团队都不陌生。本文将分享我们在内部 AI 答疑助手项目中，如何系统性地解决这类问题。文章聚焦在我们认为真正有技术深度的三个方向：基于思维链驱动的意图识别与并行检索架构、从 GraphRAG 到 LightRAG 的检索演进，以及评测体系的闭环建设。

02 背景与问题

我们团队维护着一个面向内部开发者的 AI 答疑助手，覆盖前端研发框架、开发平台、工具链、跨端组件等领域，同时深度集成了研发平台能力，支持直连平台工具查询构建、发布、监控等状态信息。

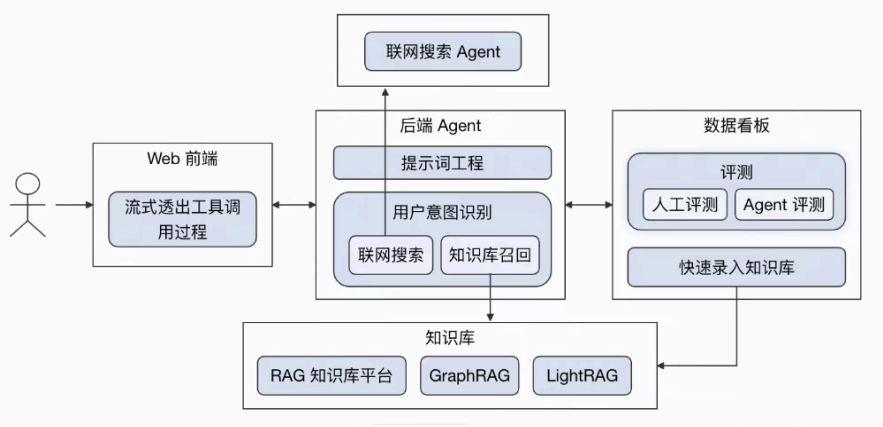
初期版本采用经典的 Naive RAG 路线：用户问题 → 知识库向量召回 → 大模型生成。上线后很快暴露出三个核心瓶颈：对提问方式高度敏感（同义改写、口语化表达导致召回不稳定）、知识碎片化（文档切分破坏上下文，多跳推理困难）、缺乏评测闭环（优化全凭直觉，没有数据驱动的迭代机制）。

我们的目标是构建一套从"问题理解 → 增强检索 → 综合回答 → 评测反馈"的全链路闭环。



03 基础能力建设（简述）

在进入核心内容之前，简要提一下我们完成的几项基础工作：用 XML 结构化语法为模型建立行为规范（工具调用分层优先级、禁止虚构内容、回复后自查清单）；引入联网搜索作为知识库的兜底补充；将模型思考过程和工具调用进度以流式方式实时透出，让答疑过程可解释；以及设计轻量级的知识录入能力降低知识库维护门槛。这些在当前的 LLM 应用开发中已是标准实践，不再展开。



04 意图识别与上下文工程：从"过一层模型"到"思维链驱动的并行检索"

为什么意图识别是核心问题

2025 年，Andrej Karpathy 公开表示他更倾向于用 **Context Engineering（上下文工程）** 来替代 Prompt Engineering 这个说法。Shopify CEO Tobi Lutke 的定义更为精确："上下文工程是为任务提供所有必要上下文，使 LLM 有能力合理解决问题的艺术。"

这个观点转变指向一个核心洞察：**LLM 应用的质量瓶颈，往往不在模型本身的能力，而在你喂给它的上下文是否足够好。** 模型拿到了正确的、完整的、结构化的上下文，它就能给出好答案；拿到了碎片化的、不相关的上下文，再强的模型也无能为力。

回到我们的答疑系统。优化提示词之后，我们发现质量瓶颈已经从"生成端"转移到了"检索端"——很多时候不是模型不会答，而是它拿到的上下文就是错的或者不全的。而根源在于，系统直接拿用户的原始问题去知识库做向量检索。用户的提问方式千差万别：口语化表达（"发版挂了"）、同义词不匹配（用户说"灰度"而文档写"分组发布"）、问题层次不清（只描述表象不触及根因）、上下文缺失（一句话提问缺少必要背景）。

这就是一个典型的上下文工程问题：**如何在用户模糊的输入和知识库精确的内容之间架起一座桥？**

现有意图识别方案的局限

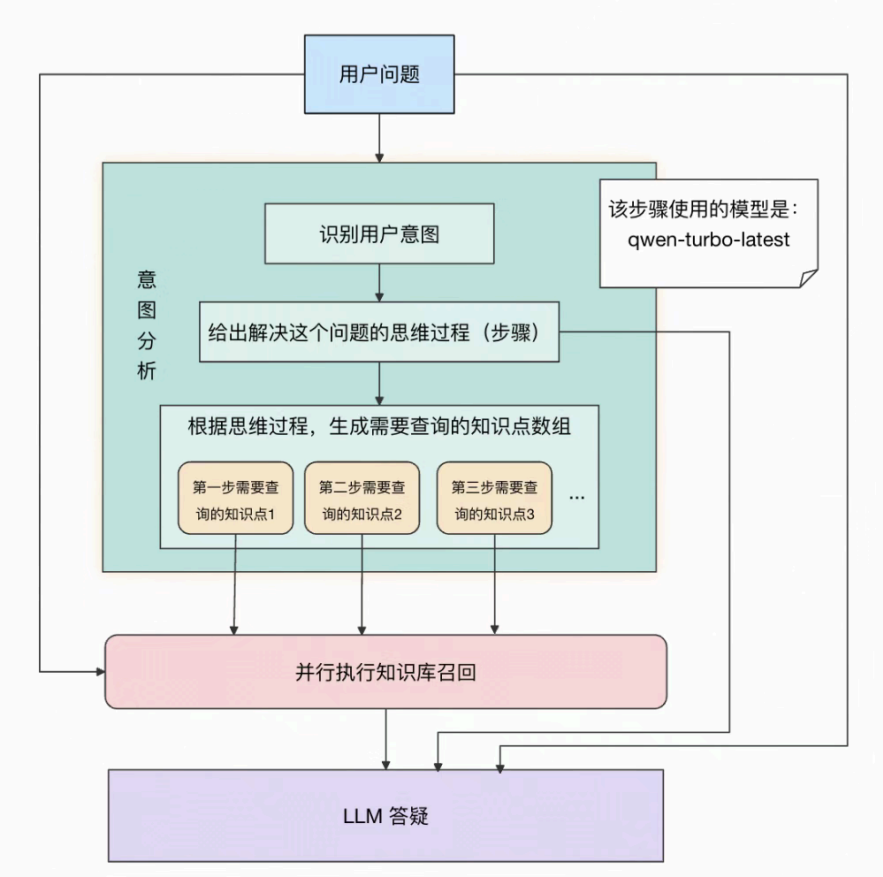
当前业界大多数 RAG 系统处理这个问题的方式是"过一层模型"——把用户的原始问题交给一个 LLM 做一次 query rewriting（查询改写），生成一个或几个优化后的查询语句，然后拿改写后的查询去检索。这种方式在简单问题上有效，但有两个本质局限。

第一，改写是"平面"的。 它只是在语义层面对原始问题做同义替换或扩展，并没有真正理解用户问题的内在结构。比如用户问"我的小程序上线后白屏了，之前本地开发都正常"，简单的 query rewriting 可能会把它改写成"小程序白屏问题排查""小程序上线后白屏原因"这类查询。但实际上这个问题包含多个维度的信息需求：本地与线上的环境差异、白屏的可能原因、渲染流程的排查步骤——这些维度之间有逻辑递进关系，而非简单并列。

第二，检索是串行的。 改写后的查询通常仍然是串行执行，一次检索拿到一批结果，结果的多样性和覆盖面受限于单次检索的能力上限。

我们的方案：CoT 驱动的查询分解与并行召回

我们设计的意图识别不是简单的"过一层模型"，而是一个**基于思维链（Chain of Thought）的多步查询分解与并行检索架构**。整个流程如下：



第一步，识别用户意图。 用一个轻量快速的模型（我们使用 qwen-turbo-latest，选择它的原因是这一步需要极低延迟，而意图识别本身不需要强推理能力）接收用户的原始问题，识别出核心意图——用户到底想解决什么问题。

第二步，生成思维过程。 这是关键的创新点。模型不是直接输出改写后的查询，而是先**给出解决这个问题的思维过程（步骤）**。比如面对"小程序上线后白屏"这个问题，模型会推理出解决这个问题大致需要经过这些步骤：首先排查线上环境与本地的差异（构建配置、CDN、域名白名单等）→ 其次检查渲染链路可能的断点（JS 加载失败、WebView 版本兼容性等）→ 最后确认监控和日志中有没有相关报错信息。

第三步，基于思维过程生成多组查询。 根据上一步推理出的每个步骤，分别生成对应的知识库查询关键词组。每个步骤可能产出多条查询——比如"排查线上环境差异"这个步骤会生成"小程序构建配置差异""CDN 资源加载失败""域名白名单配置"等查询；"检查渲染链路断点"这个步骤会生成"WebView 白屏排查""JS 资源加载异常""容器渲染异常"等查询。

第四步，并行执行知识库召回。 所有查询组**并行**发送到知识库执行向量检索，结果汇总去重后，作为完整的上下文交给最终的答疑模型（LLM）生成回答。

这个架构的核心区别在于两点。第一，**它引入了"思维链"作为查询分解的骨架**——不是随机地扩展关键词，而是沿着"解决问题的逻辑步骤"来组织检索，使得召回的知识天然具有逻辑递进关系，模型在生成答案时能拿到一个完整的"解题思路"而非一堆零散片段。第二，**多组查询并行执行**，大幅提升了召回的覆盖面和效率——传统的串行查询改写可能产出 2–3 条查询，而

我们的方案可以一次性产出十几条覆盖不同维度的查询，并行执行后的召回率远高于单次检索。

与上下文工程的深层关联

用上下文工程的视角重新审视这个架构，它实际上是在做一件事：**在用户提问和 LLM 生成之间，动态构建最优的上下文。**

传统 RAG 的上下文构建是"被动"的——用户问什么就检索什么，检索到什么就塞什么，上下文的质量完全取决于用户提问的质量和向量检索的运气。而我们的方案是"主动"的——系统先理解问题的结构，规划出需要哪些维度的信息，然后有目的地去检索和组装。

Karpathy 说的"上下文工程"的核心就在这里：**不是在写更好的提示词，而是在为 LLM 精心策划它在推理时需要看到的一切信息。** 我们的意图识别 + 并行检索架构，本质上就是一个面向答疑场景的上下文工程实现——用 CoT 规划上下文的结构，用并行检索填充上下文的内容，确保最终交给模型的上下文既完整又有条理。

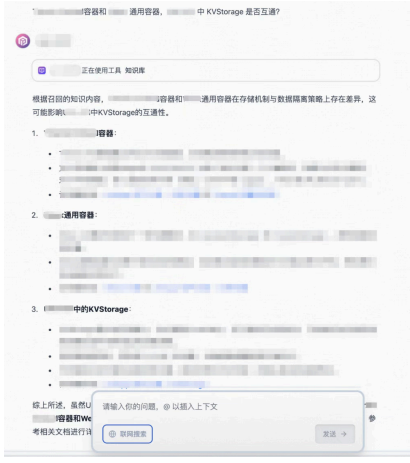
效果对比

下面是两组实际案例的对比。第一组是直接用原始问题检索后的回答效果——可以看到回答虽然沾边，但缺少关键步骤，且存在泛泛而谈的问题。第二组是经过 CoT 意图识别和并行召回后的效果。

未使用意图识别前



使用意图识别后



意图识别解决了"用什么去检索"的问题，大幅提升了召回的准确率和覆盖面。但它无法解决检索架构本身的结构性局限——即便查询再精准，如果知识库中的内容本身是碎片化的、缺乏关联的，召回结果的质量仍然有天花板。

这就引出了我们探索的第二个核心方向：从 GraphRAG 到 LightRAG 的检索架构演进。

传统向量 RAG 的核心流程是：文档切块 → 向量化存储 → 查询时做 embedding 相似度匹配 → 将匹配到的 chunk 拼接后交给模型。即便有了我们的意图识别做查询增强，这个流程本身仍有三个结构性的局限。

第一，上下文在切块时被物理性地切断了。 一篇文档讲述了从"配置 → 构建 → 部署"的完整流程，切成 chunk 后每一块都只包含其中一个步骤的片段。用户问"我配置好了但部署失败了怎么办"，系统可能只召回了"部署"相关的 chunk，而缺少"配置"部分的前置信息，导致模型给出的排查建议不完整。

第二，无法进行跨文档的关联推理。 当用户的问题需要综合多篇文档的信息才能回答时——比如"A 组件和 B 组件在事件通信机制上有什么区别"——传统 RAG 只能分别找到 A 和 B 的相关片段，但无法将它们的关系显式地关联起来。模型只能基于拼接在一起的独立片段"猜测"关联，质量高度不稳定。

第三，知识以"碎片"而非"结构"的形态存在。 在传统 RAG 中，同一个技术概念可能出现在十几篇文档中，但系统并不知道这些描述指向的是同一个实体。每次检索都是在一个"扁平"的向量空间中做近似匹配，无法利用知识之间的结构化关系。

这些问题存在于检索架构层面，需要在知识的表示和组织方式上做出结构性改变。

06 GraphRAG：用知识图谱重构检索

核心思路

GraphRAG 是微软在 2024 年提出的方法[1]，核心思路可以用一句话概括：**在 RAG 流程中引入知识图谱，将"基于文本片段的平面检索"升级为"基于实体–关系网络的结构化检索"。**

如果传统 RAG 是"在一堆散落的卡片中翻找相关的那几张"，GraphRAG 就是"先把这些卡片的内容关系整理成一张思维导图，然后沿着脉络去找答案"。

索引构建：从文档到图谱

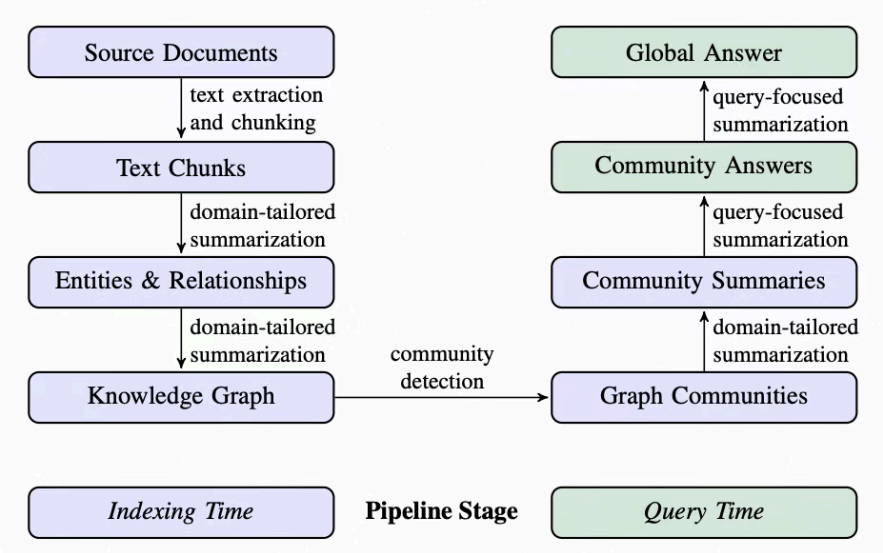
GraphRAG 的索引构建是一个多阶段流水线，远比传统 RAG 的"切块 + embedding"复杂得多。

第一步，文本分块与实体抽取。 将原始文档切分为文本单元（TextUnit），然后用 LLM 对每个文本单元做实体识别和关系抽取。比如从一段描述中提取出 `(WebView, 是, 渲染容器)`、`(WebView, 支持, 离线包加载)` 这样的三元组。这一步的质量直接决定了后续图谱的质量，也是 GraphRAG 高度依赖 LLM 的原因之一。

第二步，图构建与合并。 将所有抽取出的实体和关系合并成一张全局知识图谱。来自不同文档的相同实体会被合并为同一个节点，它们之间的关系也会被汇总。这一步解决了传统 RAG 中"信息孤岛"的问题——原本散落在各个文档中的零散信息，现在通过实体节点被关联在了一起。

第三步，社区发现（Community Detection）。 这是 GraphRAG 最独特的设计。它使用 **Leiden 算法**对知识图谱做层次化聚类，将紧密关联的实体分组为"社区"（Community）。Leiden 算法是 Louvain 算法的改进版，能保证每个社区内部是连通的，避免产生"断裂社区"。聚类结果是一个多层级的社区树：顶层是覆盖全局的大主题（如"跨端渲染架构"），底层是具体的技术细节（如"WebView 离线包加载策略"）。

第四步，社区摘要生成。 对每个社区，用 LLM 生成一份结构化的摘要报告。这份报告总结了社区中包含哪些核心实体、它们之间的关键关系、以及整体的主题概述。社区摘要是 GraphRAG 处理"全局性问题"的关键——当用户问一个需要鸟瞰式回答的问题时，系统不必遍历所有原始文本，只需检索相关的社区摘要即可。



查询引擎：Local Search 与 Global Search

GraphRAG 提供两种查询模式，分别应对不同类型的问题。

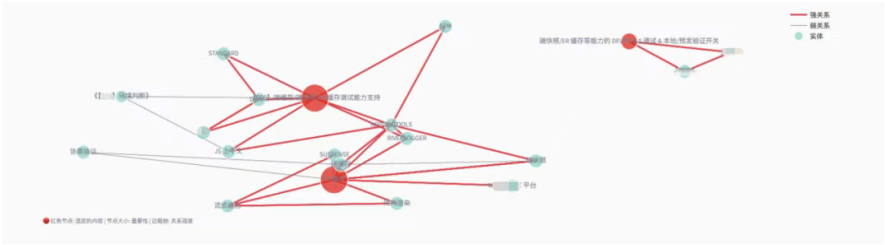
Local Search（局部搜索） 适用于具体的、有明确目标的问题，比如"WebView 的离线包加载失败怎么排查"。它的流程是：先通过 embedding 找到与问题最相关的实体节点，然后沿着图谱的边扩展到关联的实体和关系，再加上这些实体对应的原始文本片段和社区报告，一起

作为上下文交给模型。相比传统 RAG，它多了"沿图谱脉络扩展"这一步，因此能拿到更完整的上下文。

Global Search（全局搜索） 适用于宏观的、需要综合概括的问题，比如"我们的跨端方案整体架构是什么样的"。它不走传统的向量检索路径，而是直接利用社区摘要：将所有相关社区的摘要作为输入，让模型做 Map-Reduce 式的汇总——先对每个社区摘要生成部分答案（Map），再将部分答案合并成最终回答（Reduce）。这种模式在传统 RAG 中是无法实现的，因为传统 RAG 缺少社区层级的知识组织。

实验验证与工程困境

我们选取知识库中某模块的文档做了实验，使用 Streamlit 对构建出的知识图谱做了可视化。



GraphRAG 结果可视化

效果方面，GraphRAG 确实兑现了它的承诺。 在复杂问答和全局理解场景下，回答质量有显著提升。特别是需要跨多篇文档综合信息的问题，GraphRAG 通过社区摘要给出的回答明显比传统 RAG 更全面、更有条理。

但工程化问题非常严重，以至于在我们的答疑场景中几乎不可用。 具体来说有四个致命问题：

索引构建成本极高。 每篇文档的实体抽取和关系挖掘都需要调用 LLM（通常依赖 GPT-4 级别的模型才能保证抽取质量），44 篇文档的索引构建消耗了大量 API 调用。社区摘要的生成又是一轮 LLM 调用。整个索引构建的费用不菲。

不支持增量更新。 这是最致命的问题。GraphRAG 的社区结构是通过 Leiden 算法在全局图上计算出来的，加入新文档意味着图谱结构发生变化，社区边界需要重新划分，所有社区摘要需要重新生成。对于一个知识库每天都在更新的答疑系统来说，每次更新都要重建整个图谱，这个代价是无法接受的。

查询延迟过高。 Global Search 需要遍历多个层级的社区摘要并做 Map-Reduce 聚合，一次查询可能耗时 5 分钟以上。即使是 Local Search，由于涉及图遍历和上下文拼装，延迟也显著高于传统 RAG。在实时答疑场景下，用户等不了这么久。

对抽取质量高度敏感。 如果实体抽取和关系挖掘的质量不高（使用较弱的模型、或文档本身表述不规范），构建出的知识图谱就会充满噪音，社区划分的结果也会变得不合理，最终导致检索质量反而不如传统 RAG。

GraphRAG 给了我们重要的启示——图结构确实能有效解决传统 RAG 的上下文断裂和跨文档推理问题——但它的工程化方案太重了。我们需要一个保留核心优势、同时大幅降低工程成本的替代方案。



LightRAG：图增强检索的工程化落地

设计哲学：做减法

LightRAG[2] 的设计哲学可以概括为一个词——**做减法**。它的作者观察到 GraphRAG 中最昂贵的部分（社区发现 + 社区摘要）并不是在所有场景下都必需的，尤其是在私域知识库问答这种场景中，大部分问题都是具体的、局部的，真正需要"全局鸟瞰"的查询占比很低。

基于这个洞察，LightRAG 做了一个关键的架构决策：**去掉社区机制，用更直接的图索引 + 向量嵌入混合方案来替代。**

索引构建：三个核心函数

LightRAG 的索引构建围绕三个核心操作展开，论文中将它们形式化为 R(·)、P(·)、D(·) 三个函数。为了便于理解，我用更直观的方式来解释。

R(·) —— 实体与关系抽取。 与 GraphRAG 类似，用 LLM 从文本中识别出实体（作为图的节点）和实体之间的关系（作为图的边）。但 LightRAG 在这一步做了简化：不追求像 GraphRAG 那样精细的实体类型分类，而是更注重抽取的效率和覆盖面。

P(·) —— 键值对生成。 这是 LightRAG 区别于 GraphRAG 的关键创新。对于每个实体节点和关系边，LightRAG 会用 LLM 生成一个 (Key, Value) 文本对。Key 是一个便于检索的短语或关键词（实体直接用名称作为 Key，关系则通过 LLM 增强生成多个全局性主题关键词）；Value 是一段总结性文本，聚合了该实体或关系在原始文档中的所有相关描述。这种设计的巧妙之处在于：**它用键值对的形式替代了 GraphRAG 的社区摘要，在不做社区聚类的情况下实现了信息的聚合和压缩。**

D(·) —— 去重与合并。 自动识别来自不同文档段落的相同实体和关系，将它们合并为同一个节点/边。这一步不仅压缩了图的规模，更关键的是它使得**增量更新变得天然可行**——新文档中的实体如果与已有图谱中的实体匹配，只需合并信息而不需要重建整个图结构。

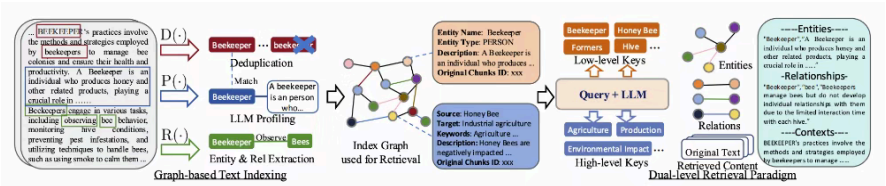


Figure 1: Overall architecture of the proposed LightRAG framework.

双层检索范式：一个查询，两种路径

LightRAG 设计了一个双层检索范式来应对不同类型的查询，这是它在保持轻量的同时不丢失 GraphRAG 多层级检索能力的关键。

低级检索（Low-Level Retrieval） 聚焦具体实体及其直接关联的属性和关系。当用户问"WebView 的离线包加载流程是什么"时，系统首先通过向量相似度找到 "WebView" 和 "离线包" 相关的实体节点，然后提取这些节点的 Value 文本以及它们之间的关系描述，拼装为上下文。这一层类似于 GraphRAG 的 Local Search，但省去了社区遍历的开销，检索路径更短。

高级检索（High-Level Retrieval） 则面向更抽象、更宏观的主题性查询。它不直接检索实体节点，而是检索关系边上的全局性主题关键词（这些主题关键词是在 P(·) 函数中由 LLM 生成的）。比如用户问"跨端方案的整体技术选型思路是什么"，系统会匹配到多条关系边上的高层主题（如"跨端渲染策略""容器化方案选型"等），再沿这些边关联到相关实体，汇总成全局性的上下文。

在实际查询中，LightRAG 会同时触发两层检索，合并去重后交给模型生成答案。这种设计确保了无论问题是具体的还是抽象的，都能获得有效的上下文覆盖。

增量更新：工程化的关键优势

在实际的答疑系统中，知识库不是静态的——每天都可能有新文档加入、旧文档更新。这使得"增量更新"成为衡量一个检索架构是否真正可工程化的核心指标。

GraphRAG 在这方面几乎是灾难性的。它的社区结构是在全局图上计算出来的，任何图谱变动都可能导致社区边界的重新划分，进而需要重新生成所有受影响社区的摘要报告。实际操作中，大多数团队选择定期全量重建，成本高昂且实时性差。

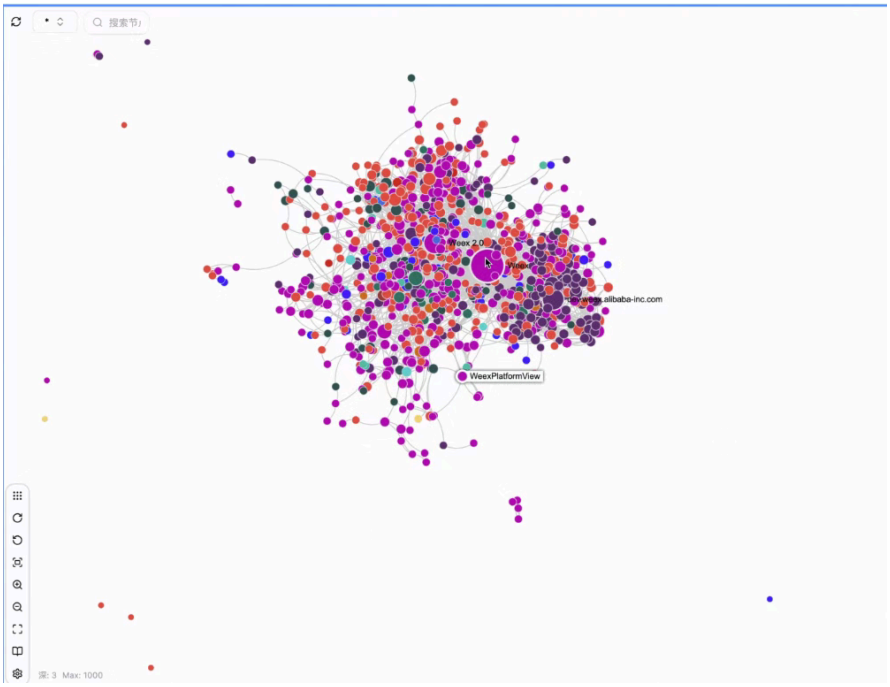
LightRAG 之所以能支持增量更新，根本原因在于**它没有社区层级的依赖**。新文档加入时，只需执行三步：对新文档运行 R(·) 抽取实体和关系 → 通过 D(·) 与已有图谱去重合并 → 对新

增/更新的节点和边执行 P(·) 生成键值对。整个过程是局部的、增量的，不会影响已有图谱中其他部分的结构和索引。

在我们的实测中，向一个已有的知识图谱中新增文档，LightRAG 只需要处理新增部分的实体和关系，耗时和 API 调用量都远小于全量重建。

实验对比

我们用同一批 44 篇文档分别在 GraphRAG 和 LightRAG 上做了实验。



从实验结果来看：**LightRAG 在索引构建速度上比 GraphRAG 快数倍**（省去了社区发现和社区摘要生成的开销），**查询延迟从分钟级降到了秒级，API 调用量和 token 消耗大幅减少**。在回答质量上，对于大部分具体问题，LightRAG 与 GraphRAG 的 Local Search 表现接近；在全局性问题上，LightRAG 的高级检索虽然不如 GraphRAG 的 Global Search 那样有社区摘要的加持，但在我们的场景中已经足够用了——毕竟用户问的 80% 以上都是具体的技术问题，而非需要鸟瞰全局的综合性提问。

权衡总结：GraphRAG 适合对回答质量有极致要求且能承担高成本的离线分析场景；LightRAG 则是需要实时响应、频繁更新的在线答疑场景下更务实的选择。

更前沿的方向：MiniRAG 与 Agentic RAG

值得一提的是，图增强 RAG 领域的演进仍在加速。在我们实践 LightRAG 的同时，社区又涌现了一些值得关注的新方向。

MiniRAG 尝试将图增强 RAG 的能力带到小模型（SLMs）上。它提出了异构图索引（Heterogeneous Graph Indexing）结构，同时包含文本片段节点和语义概念节点两种节点类型，通过轻量级的语义感知异构图索引和拓扑增强检索，在不依赖大模型强语义理解能力的前提下实现知识图谱增强检索，这为资源受限场景（如端侧部署）打开了新的可能。

Agentic RAG 则代表了另一个维度的演进。它不再将 RAG 视为一个固定的 pipeline，而是让 AI Agent 自主决定"是否检索、检索什么、检索结果够不够、需不需要重新查"。其中，**自适应 RAG（Adaptive RAG）** 根据问题复杂度动态选择检索策略；**自反思 RAG（Self-Reflective RAG）** 在生成答案后评估质量，不满意就重新检索；**纠正性 RAG（Corrective RAG）** 对检索结果做质量评估，剔除噪音文档后再生成。这些模式将检索从"一次性查找"进化为"多轮迭代优化"，与图增强检索可以形成互补——用图结构保证检索的结构化质量，用 Agent 机制保证检索的自适应能力。

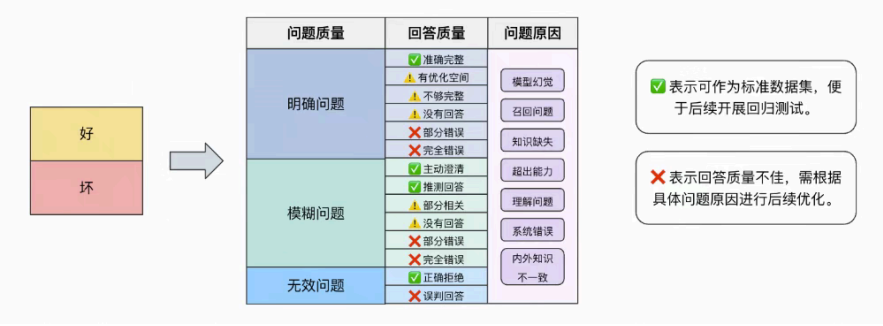
08 评测体系：让优化从玄学变成工程

从"好/坏"到多维度标注

检索架构再怎么优化，如果没有评测体系来量化效果，就只是在"凭感觉"做事。

我们的评测体系经历了一次关键迭代。初版只有"好"和"坏"两个标签，用了一段时间后发现粒度太粗——一个"坏"的回答，可能是知识库缺内容、可能是检索没命中、可能是模型幻觉，但标签无法区分这些不同的根因。

于是我们引入了**多维度标注体系**，从三个维度评估：**问题质量**（用户问题本身是否清晰）、**回答质量**（准确性和完整度）、**问题成因**（根因是什么）。有了这个体系，每个 Bad Case 都能被归因到具体的环节，对症下药：幻觉就强化提示词约束，召回不足就调检索策略，知识缺失就推动补录，超出能力就引导转人工。



评测 Agent 的"过度自信"问题

我们还做了一个有意思的探索——用一个 Agent 自动评测每条回答的质量，辅助人工标注。

结果发现了一个值得警惕的现象：**评测 Agent 存在显著的"乐观偏差"。** 它判定为"需改进"的，人工几乎都会确认为 Bad Case（精准率高）；但它判定为"优秀"的，人工仍有相当比例会标为"坏"（召回率低）。

这意味着**模型在评估自身（或同类模型）的输出时，倾向于过度自信**——在证据不足或知识覆盖不全的情况下，模型往往无法意识到自己"不知道"，反而会给出看似合理的高分。这个发现对架构设计有直接影响：在任何 LLM 应用中，都不能完全依赖模型的自我评估，人工校验环节不可或缺。后续我们计划将评测 Agent 与多维度标注体系对齐，通过更精细的评分维度来约束模型的乐观倾向。

09 总结与展望

回顾整个优化历程，我们最深的体会是：**AI 答疑系统的质量，取决于你为模型构建上下文的能力——这正是"上下文工程"的核心命题。**

我们的三个核心优化方向，本质上都是在做同一件事：为 LLM 构建更好的上下文。基于 CoT 的意图识别解决了"用什么去检索"的问题——通过思维链分解和并行召回，让检索的覆盖面和精准度大幅提升，这是上下文工程在查询侧的实践。从 GraphRAG 到 LightRAG 的演进解决了"检索到的知识如何组织"的问题——通过图结构让知识之间的关联变得显式可查，这是上下文工程在知识侧的实践。而评测体系则提供了衡量上下文质量的度量工具，让整个优化过程可量化、可迭代。

这三者互为补充：意图识别产出精准的多路查询，图增强检索返回结构化的关联知识，评测闭环驱动两者持续改进。它们共同构成了一套面向答疑场景的上下文工程方案。

从传统 RAG 到 GraphRAG 再到 LightRAG，本质上是知识表示方式的升级：从扁平的向量空间，到结构化的实体–关系图谱。GraphRAG 验证了图结构增强检索的有效性，LightRAG 则证明了这条路线可以在工程上落地。而 Agentic RAG 和 MiniRAG 等新方向的出现，预示着检索架构还将继续演进——未来的 RAG 系统可能不再是一个固定的 pipeline，而是一个能自主决策检索策略的智能体。我们的 CoT 驱动意图识别，某种程度上已经具备了 Agentic RAG 中"自主规划检索策略"的雏形。

10 参考文献

1. Edge D, Trinh H, Cheng N, et al. From local to global: A graph RAG approach to query–focused summarization. arXiv preprint arXiv:2404.16130, 2024.
2. Guo Z, Xia L, Yu Y, et al. LightRAG: Simple and fast retrieval–augmented generation. arXiv preprint arXiv:2410.05779, 2024.

11 团队介绍

本文作者凡烟，来自淘天集团–跨端技术团队。我们是淘天集团核心基础设施团队，立足淘宝，专注于构建世界级的多端技术平台，为数亿用户提供极致流畅的购物体验。在AI重塑终端体验与研发范式的浪潮中，我们正驱动终端智能化创新、构建AI赋能的研发新基建，寻找兼具工程深度与AI思维的伙伴共同定义下一代终端体验。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 1.1万

留言

写留言

还没有留言，来写首评